



Linux Fundamentals

0%



Section 21 / 30

# Backup and Restore

Linux systems provide a range of powerful tools for backing up and restoring data, designed to be both efficient and secure. These tools help ensure that our data is not only protected from loss or corruption, but also easily accessible when we need it.

When backing up data on an Ubuntu system, we have several options, including:

- Rsync
- Deja Dup
- Duplicity

Rsync is an open-source tool that allows for fast and secure backups, whether locally or to a remote location. One of its key advantages is that it only transfers the portions of files that have changed, making it highly efficient when dealing with large amounts of data. Rsync is particularly useful for network transfers, such as syncing files between servers or creating incremental backups over the internet.

Duplicity is another powerful tool that builds on Rsync, but adds encryption features to protect the backups. It allows you to encrypt your backup copies, ensuring that sensitive data remains secure even if stored on remote servers, FTP sites, or cloud services like Amazon S3. Duplicity provides an extra layer of security while maintaining Rsync's efficient data transfer capabilities.

Think of your data as valuable treasures stored in a house. The backup tools on Linux, such as Rsync, Duplicity, and Deja Dup, act like different kinds of safes. Rsync is like a fast-moving transport that only carries what's new or changed, making it the ideal way to send updates to a remote vault. Duplicity is a high-security safe that not only stores the treasure but also locks it with a complex code, ensuring no one else can access it. Deja Dup is a simple, accessible safe that anyone can operate, while still offering the same level of protection. Encrypting your backups adds an additional lock on your safe, ensuring that even if someone finds it, they can't get inside.

For users who prefer a simpler, more user-friendly option, Deja Dup offers a graphical interface that makes the backup process straightforward. Behind the scenes, it also uses Rsync, and like Duplicity, it supports encrypted backups. Deja Dup is ideal for users who want quick, easy access to backup and restore options without needing to dive into the command line.

Ensuring the security of your backups is just as important as creating them. Encrypting your backup data helps safeguard it from unauthorized access, providing peace of mind that sensitive information remains protected. On Ubuntu systems, you can use additional encryption tools like GnuPG, eCryptfs, or LUKS to add another layer of protection to your backups.

Backing up and restoring data is an essential practice for anyone working with Ubuntu. By using tools like Rsync, Duplicity, and Deja Dup, you can ensure that your data is securely stored and easily retrievable, giving you confidence that, in case of an unexpected data loss, your information can be restored quickly and reliably.

In order to install Rsync on Ubuntu, we can use the `apt` package manager:

## Install Rsync

```
athane@htb[/htb]$ sudo apt install rsync -y
```

shellsession

This will install the latest version of Rsync on the system. Once the installation is complete, we can begin using the tool to back up and restore data. To backup an entire directory using `rsync`, we can use the following command:

## Rsync - Backup a local Directory to our Backup-Server

```
athane@htb[/htb]$ rsync -av /path/to/mydirectory  
user@backup_server:/path/to/backup/directory
```

shellsession

This command will copy the entire directory ( `/path/to/mydirectory` ) to a remote host ( `backup_server` ), to the directory `/path/to/backup/directory` . The option `archive` ( `-a` ) is used to preserve the original file attributes, such as permissions, timestamps, etc., and using the `verbose` ( `-v` ) option provides a detailed output of the progress of the `rsync` operation.

We can also add additional options to customize the backup process, such as using compression and incremental backups. We can do this like the following:

shellsession

```
athane@htb[/htb]$ rsync -avz --backup --backup-dir=/path/to/backup/folder --delete  
/path/to/mydirectory user@backup_server:/path/to/backup/directory
```

With this, we back up the `mydirectory` to the remote `backup_server`, preserving the original file attributes, timestamps, and permissions, and enabled compression ( `-z` ) for faster transfers. The `--backup` option creates incremental backups in the directory `/path/to/backup/folder`, and the `--delete` option removes files from the remote host that is no longer present in the source directory.

If we want to restore our directory from our backup server to our local directory, we can use the following command:

## Rsync - Restore our Backup

```
athane@htb[/htb]$ rsync -av user@remote_host:/path/to/backup/directory  
/path/to/mydirectory
```

shellsession

## Encrypted Rsync

To ensure the security of our `rsync` file transfer between our local host and our backup server, we can combine the use of SSH and other security measures. By using SSH, we are able to encrypt our data as it is being transferred, making it much more difficult for any unauthorized individual to access it. Additionally, we can also use firewalls and other security protocols to ensure that our data is kept safe and secure during the transfer. By taking these steps, we can be

confident that our data is protected and our file transfer is secure. Therefore we tell `rsync` to use SSH like the following:

## Secure Transfer of our Backup

```
athane@htb[/htb]$ rsync -avz -e ssh /path/to/mydirectory  
user@backup_server:/path/to/backup/directory
```

shellsession

The data transfer between our local host and the backup server occurs over the encrypted SSH connection, which provides confidentiality and integrity protection for the data being transferred. This encryption process ensures that the data is protected from any potential malicious actors who would otherwise be able to access and modify the data without authorization. The encryption key itself is also safeguarded by a comprehensive set of security protocols, making it even more difficult for any unauthorized person to gain access to the data. In addition, the encrypted connection is designed to be highly resistant to any attempts to breach security, allowing us to have confidence in the protection of the data being transferred.

## Auto-Synchronization

To enable auto-synchronization using `rsync`, you can use a combination of `cron` and `rsync` to automate the synchronization process. Scheduling the cron job to run at regular intervals ensures that the contents of the two systems are kept in sync. This can be especially beneficial for organizations that need to keep their data synchronized across multiple machines. Furthermore, setting up auto-synchronization with `rsync` can be a great way to save time and effort, as it eliminates the need for manual synchronization. It also helps to ensure that the files and data

stored in the systems are kept up-to-date and consistent, which helps to reduce errors and improve efficiency.

Therefore we create a new script called `RSYNC_Backup.sh`, which will trigger the `rsync` command to sync our local directory with the remote one. However, because we are using a script to perform SSH for the rsync connection, we need to configure key-based authentication. This is to bypass the need to input our password when connecting with SSH.

First, we generate a key pair for our user.

```
shellsession  
  
athane@htb[/htb]$ ssh-keygen -t rsa -b 2048
```

Follow the prompts to specify the location (default is `~/.ssh/id_rsa`) and optionally provide a passphrase (leave it empty for no passphrase). Then, we need to copy our public key to the remote server.

```
shellsession  
  
athane@htb[/htb]$ ssh-copy-id user@backup_server
```

Now, we can create our script to automate the rsync backup.

## **RSYNC\_Backup.sh**

```
bash
```

```
#!/bin/bash
```

```
rsync -avz -e ssh /path/to/mydirectory user@backup_server:/path/to/backup/directory
```

Then, in order to ensure that the script is able to execute properly, we must provide the necessary permissions. Additionally, it's also important to make sure that the script is owned by the correct user, as this will ensure that only the correct user has access to the script and that the script is not tampered with by any other user.

```
athane@htb[/htb]$ chmod +x RSYNC_Backup.sh
```

shellsession

After that, we can create a crontab that tells **cron** to run the script every hour at the 0th minute.

```
athane@htb[/htb]$ crontab -e
```

shellsession

We can adjust the timing to suit our needs. To do so, the crontab needs the following content:

## Auto-Sync - Crontab

```
0 * * * * /path/to/RSYNC_Backup.sh
```

shellsession

With this setup, `cron` will be responsible for executing the script at the desired interval, ensuring that the `rsync` command is run and the contents of the local directory are synchronized with the remote host.

We encourage you to try out `rsync` using Pwnbox. Instead of syncing files with a remote server, you can use Pwnbox as both your source and destination, which makes testing simpler. To do this, create two directories on Pwnbox:

1. `to_backup` (where your original data is stored) and another called
2. `synced_backup` (where the synchronized data will be copied)

You will then transfer the data from the `to_backup` directory to the `synced_backup` directory using `rsync`. To automate this process, set up a `cron` job that runs every minute to ensure continuous synchronization. Remember, because we are testing this locally, we can use the loopback IP address 127.0.0.1 as the address for the "remote" host.

## Connect to HTB

